MODULE 23 · DOMAIN 5 - INCIDENT AND EVENT RESPONSE

Troubleshooting Failed Deployments and Operations

A method for diagnosis, where each service records its failures, the error signatures worth recognising, and how to untangle access denied.

DOMAIN 5 of 6 EXAM WEIGHT 14% MODULE 23 of 34

admin@sthithapragna.com

Troubleshooting is **narrowing**: find the layer, find the **first** failure, and read what it **actually says**.

A diagnostic method

01
Place it
Build, deploy, run, or respond?

02
First failure
The earliest error, not the loudest.

03
Read it
The exact message names the cause.

04
What changed?
Deployments, CloudTrail, Config.

05
Fix and prevent
Then add a check so it cannot recur.

ON THE EXAM

Most incidents follow a change. 'What changed just before?' is often the fastest question.

Where each service records its failures

SERVICE	LOOK AT
CodePipeline	Action execution details, linking to the underlying service
CodeBuild	Build logs and phase details
CodeDeploy	Lifecycle events per instance; agent logs on the instance
CloudFormation	Stack events: the first FAILED status reason
ECS	The stopped task's reason; service events
Auto Scaling	Activity history
Lambda	CloudWatch Logs; Errors and Throttles metrics

ON THE EXAM

A pipeline only reports that an action failed. The reason lives in the service the action called.

CloudFormation failure signatures

YOU SEE	IT USUALLY MEANS
Many 'Resource creation cancelled'	Not the cause: find the earlier failure
'already exists'	A fixed name collides with an existing resource
'Received 0 SUCCESS signals'	cfn-signal never reached CloudFormation
'Requires capabilities: CAPABILITY_IAM'	The template creates IAM resources; acknowledge it
Export 'cannot be updated as it is in use'	Another stack imports it
UPDATE_ROLLBACK_FAILED	Fix the cause, then continue rollback, skipping resources

ON THE EXAM

When one resource fails, CloudFormation cancels everything in progress. The cancellations are noise.

Pipeline and build failure signatures

YOU SEE	IT USUALLY MEANS
'Cannot connect to the Docker daemon'	Privileged mode is off
Timeouts downloading packages in a VPC build	No NAT route or endpoints
Secret fetch fails at build start	Service role lacks the fetch or kms:Decrypt permission
Cross-account action 'access denied' on artifacts	Bucket policy or customer managed key policy
Lambda action stuck, then times out	The function never reported a job result
Tests failed, build passed	An exit code was swallowed

ON THE EXAM

Every row here was a scenario in Domain 1. Troubleshooting is recognition of patterns you have already met.

CodeDeploy failure signatures

YOU SEE	IT USUALLY MEANS
No instances found for the deployment group	Tag filter or group name does not match the instances
Every deployment fails at ApplicationStop	The last successful revision's stop script is broken
Too many individual instances failed	Minimum healthy hosts could not be kept
Script timed out at a lifecycle event	The script hangs, or its timeout is too short
'file already exists at location'	Files placed outside CodeDeploy; set file-exists behaviour
Instance never picks up the deployment	Agent not running, or no instance profile access

ON THE EXAM

Read which lifecycle event failed. Its name tells you whether the problem is yours or the environment's.

ECS failure signatures

YOU SEE	IT USUALLY MEANS
'unable to place a task'	No instance has the CPU, memory or free port needed
CannotPullContainerError	Execution role lacks ECR access, no route to ECR, or wrong tag
ResourceInitializationError	Secrets or log setup failed: execution role or network path
Essential container exited	The application crashed: read its logs
Tasks replaced in a loop	Failing health checks, often too short a grace period
Deployment rolled back by itself	The circuit breaker or a deployment alarm acted

ON THE EXAM

Anything that fails before your code runs is the execution role or the network. After, it is your code or the task role.

Task execution role, or task role?

TASK EXECUTION ROLE

Used by ECS, before your code runs

- Pulls images from ECR
- Fetches secrets for environment variables
- Sends logs through the awslogs driver
- Fails as: task never starts

TASK ROLE

Used by your application

- Calls S3, DynamoDB, SQS, and so on
- Credentials appear inside the container
- Nothing to do with starting the task
- Fails as: app runs, then gets access denied

THE TIEBREAKER

Task will not start: execution role. Task runs but the app's AWS calls fail: task role.

Auto Scaling and EC2 failure signatures

ACTIVITY HISTORY SAYS	IT USUALLY MEANS
Insufficient capacity for the instance type	Too narrow a choice: add types and zones
Subnet has no free IP addresses	The subnet is too small for the group
Instance limit or quota exceeded	Request a quota increase
Invalid AMI, key pair or launch template	A reference points to something missing
Instances launched then terminated repeatedly	Health checks fail before the app is ready

ON THE EXAM

Capacity errors for one instance type in one zone: a mixed instances policy across types and zones.

Lambda failure signatures

YOU SEE	IT USUALLY MEANS
429 throttling errors	Concurrency exhausted, or scaling rate exceeded
'Task timed out' calling AWS APIs from a VPC	No NAT route or VPC endpoint
AccessDeniedException in the logs	The execution role lacks the permission
Runtime exited, memory limit reached	Needs more memory
Init errors on every cold start	Failing code outside the handler, or a missing dependency

ON THE EXAM

Attaching a function to private subnets removes its internet path. AWS APIs then need NAT or endpoints.

Untangling access denied

- Most error messages now say **which kind of policy** denied: SCP, boundary, session, identity or resource policy.
- CloudTrail records the failed call, its principal, and the error code.
- The **IAM policy simulator** tests a principal's policies against an action and resource.
- 'Explicit deny in a service control policy': no identity policy change can fix it.

> ON THE EXAM
>
> Read the whole error message first. It usually names the policy type, which tells you where to look.

The layers that must all allow

01
Organisation
SCPs and RCPs: no explicit deny, and an allow.

›

02
Boundaries
Permissions boundary, session policy.

›

03
Identity
The principal's own policies allow it.

›

04
Resource
Bucket, key, queue or endpoint policies agree.

›

05
Network
VPC endpoint policies, and a route at all.

ON THE EXAM

Admin rights inside the account cannot get past a ceiling set above it, or a resource that refuses.

Asking 'what changed?'

- **Deployment history**: pipelines, CodeDeploy and ECS deployments around the start time.
- **CloudTrail**: who changed which resource, and when.
- **Config**: what a resource's configuration was before and after.
- **AWS Health**: whether AWS itself had an event affecting your resources.

> ON THE EXAM
>
> No deployment, sudden failure: look for a manual change in CloudTrail or Config before blaming the application.

> A stack creation fails. The events list shows twelve resources with 'Resource creation cancelled'. Earlier, one event shows a Lambda function failing: 'The role defined for the function cannot be assumed by Lambda'.
>
> The execution role in the template trusts ec2.amazonaws.com.

What should the DevOps engineer fix?

A. Change the role's trust policy to trust lambda.amazonaws.com

B. Increase the creation timeouts on the twelve cancelled resources

C. Add DependsOn attributes to every resource

D. Create the stack with rollback disabled and try again

◆ PAUSE AND DECIDE

What should the DevOps engineer fix?

A Trust lambda.amazonaws.com

The first failure says Lambda cannot assume the role; the trust names the wrong service.

B Longer timeouts

The cancellations are consequences, not causes.

C DependsOn everywhere

Ordering was not the problem. The role was never assumable.

D Disable rollback

Keeps the broken state for inspection; the same error recurs.

THE SIGNAL

Twelve loud cancellations, one quiet first failure. Fix the first; the rest follow.

> New ECS tasks on Fargate never start. The stopped reason reads: 'ResourceInitializationError: unable to pull secrets ... AccessDeniedException ... secretsmanager:GetSecretValue'.
>
> The task definition injects a database password from Secrets Manager as an environment variable. The task role allows secretsmanager:GetSecretValue.

What should the DevOps engineer fix?

A. Add secretsmanager:GetSecretValue to the task role again

B. Allow secretsmanager:GetSecretValue, and kms:Decrypt if needed, on the task execution role

C. Open the task's security group to all inbound traffic

D. Increase the task's memory

◆ PAUSE AND DECIDE

What should the DevOps engineer fix?

A **Task role again**

The task role is used by the app after start. Injecting secrets happens before.

B **Task execution role**

ECS fetches injected secrets before the code runs, using the execution role.

C **Open the security group**

Inbound rules do not affect outbound secret retrieval, and the error is access denied.

D **More memory**

The task never started. Memory is irrelevant.

THE SIGNAL

Fails before the code runs: execution role. The planted detail is that the task role already had the permission.

> A CodeDeploy deployment to EC2 fails immediately with 'No instances found for your deployment group'. The deployment group targets instances tagged Env=prod.
>
> Last week the team moved to a new launch template, whose instances are tagged Environment=prod. The agent is running on every instance.

What should the DevOps engineer do?

A. Reinstall the CodeDeploy agent on every instance

B. Lower the deployment configuration's minimum healthy hosts

C. Make the deployment group's tag filter and the instances' tags match

D. Add permissions to the CodeDeploy service role

◆ PAUSE AND DECIDE

What should the DevOps engineer do?

A Reinstall agent

The scenario says the agent runs. Targeting fails before any agent is involved.

B Minimum healthy hosts

There are zero targets. No percentage of zero helps.

C Match the tags

Env and Environment are different keys, so the filter matches nothing.

D Service role

A permissions problem would say access denied, not no instances found.

THE SIGNAL

'No instances found' plus a recent change to tags or templates: the target filter no longer matches.

> A Lambda function that writes to DynamoDB was attached to private subnets so it could also reach an internal database. Since then, every invocation fails with 'Task timed out after 30.00 seconds'.
>
> The subnets have no NAT gateway. The internal database calls succeed.

What should the DevOps engineer do?

A Increase the function timeout to 5 minutes

B Increase the function's memory

C Set reserved concurrency on the function

D Add a DynamoDB gateway VPC endpoint to the subnets' route tables

◆ PAUSE AND DECIDE

What should the DevOps engineer do?

A Longer timeout

The call can never succeed without a route. It would just fail more slowly.

B More memory

Speed is not the problem. There is no path.

C Reserved concurrency

Unrelated to network paths.

D DynamoDB gateway endpoint

Gives private subnets a route to DynamoDB, with no NAT needed and no data charge.

THE SIGNAL

Timed out only since joining a VPC, calls to an AWS service: no route. Gateway endpoint for DynamoDB or S3.

> A developer with the AdministratorAccess policy tries to create an RDS database in eu-south-2 and receives: 'with an explicit deny in a service control policy'.
>
> The developer asks for more permissions to be added to their IAM role.

What should the DevOps engineer do?

A Explain the organisation restricts Regions; use an allowed Region, or request an exception to the SCP

B Attach an additional policy allowing rds:* to the developer's role

C Remove the developer's permissions boundary

D Add the developer to the RDS KMS key's key policy

◆ PAUSE AND DECIDE

What should the DevOps engineer do?

A It is the SCP

The message names an explicit deny in an SCP. Only the organisation's policy, or the Region, can change.

B More IAM permissions

The developer already has administrator access. An explicit deny beats any allow.

C Remove boundary

The message names an SCP, not a boundary.

D Key policy

The message names an SCP, not a key.

THE SIGNAL

Read the policy type in the message. 'Explicit deny in a service control policy' is a ceiling, not a missing grant.

> An Auto Scaling group uses one GPU instance type in a single Availability Zone. During a traffic surge, activity history shows repeated 'insufficient capacity' errors, and no instances launch.
>
> The application runs equally well on several similar instance types.

What should the DevOps engineer do?

A Wait and retry the scale-out later

B Use a mixed instances policy with several suitable types, across several Availability Zones

C Request a higher EC2 instance quota

D Switch to a different AMI

◆ PAUSE AND DECIDE

What should the DevOps engineer do?

A **Wait**

Surges are exactly when capacity is scarce. Waiting fails users now.

B **More types, more zones**

Many more capacity pools to draw from, and the app runs on any of them.

C **Higher quota**

A quota error says 'limit exceeded'. This is AWS lacking that type in that zone.

D **Different AMI**

The image does not change hardware availability.

THE SIGNAL

Insufficient capacity is not a quota. Widen the choice: instance types and Availability Zones.

> At 14:05, an application's calls to its database started failing with connection timeouts. The pipeline shows no deployment for three days, and the database is healthy.
>
> The on-call engineer wants the fastest way to find the cause.

What should the engineer check first?

A Read the application's debug logs from the last three days

B Enable X-Ray tracing and wait for new traces

C CloudTrail and the Config timeline for changes to the relevant security groups and route tables around 14:00

D Restart the application servers

◆ PAUSE AND DECIDE

What should the engineer check first?

A Three days of logs

Slow, and the app did not change. Timeouts suggest the network did.

B Enable X-Ray

Tells you where time goes, later. Not what changed at 14:05.

C CloudTrail and Config around 14:00

A sudden failure with no deployment points to a manual change; these show who changed what.

D Restart

Acting before diagnosing, and it will not fix a changed network rule.

THE SIGNAL

Sudden failure, exact start time, no deployment: ask what changed. Timeouts point to the network.

> An ECS service on EC2 uses bridge networking, with container port 80 mapped to host port 80. Scaling from 4 to 8 tasks leaves 4 tasks pending on a cluster with plenty of spare CPU and memory.
>
> Service events say no container instance met the task's requirements, citing a port already in use.

What should the DevOps engineer change?

A. Increase the memory reserved for each task

B. Use larger container instances

C. Add a placement constraint that spreads tasks across Availability Zones

D. Use dynamic host port mapping, or the awsvpc network mode, with the load balancer registering each task

◆ PAUSE AND DECIDE

What should the DevOps engineer change?

A More memory

Memory is plentiful. The port is the constraint.

B Larger instances

Still only one host port 80 per instance, however large.

C Spread across zones

Changes where tasks go, not how many fit per instance.

D Dynamic ports or awsvpc

Each task gets its own port or its own network interface, so many fit per instance.

THE SIGNAL

Spare CPU and memory, tasks pending, 'port in use': a fixed host port allows one task per instance.

Module 23, and Domain 5, in five lines

- Place it, find the first failure, read it, ask what changed, fix and prevent.
- Cancellations follow the real failure. Pipelines point to the service that failed.
- ECS: execution role before your code runs, task role after.
- Access denied messages name the policy type. SCP denies are ceilings, not missing grants.
- VPC timeouts to AWS services: missing routes. Capacity errors: widen types and zones.

> ON THE EXAM
>
> Domain 5 complete. Next, module 24 opens Domain 6: IAM at scale.